

Examination Timetabling Problem

Le Minh Duc - 202416677

June 6 2025

Abstract

The Examination Timetabling Problem (ETP) is a well-known NP-hard combinatorial optimization problem with significant practical relevance in educational institutions. It involves assigning a set of exams to available time slots and rooms while satisfying various hard and soft constraints. This study investigates five solution techniques: a Constraint Programming (CP) model, a Mixed Integer Programming (MIP) model, and three heuristic-based methods. Experimental results demonstrate that the heuristic-based approach consistently outperforms CP and MIP in terms of solution quality and computational efficiency. A comprehensive analysis of the strengths and limitations of each method is presented, along with discussions on possible improvements and future research directions. The findings contribute to the advancement of ETP-solving methodologies and provide actionable insights for practitioners aiming to optimize exam scheduling and related packing problems.

1 Introduction and Objectives

There are N classes $1, 2, \dots, N$ that need to be scheduled for the final exam. Each class must be assigned to a time-slot and a room. There are M rooms $1, 2, \dots, M$ that can be used for scheduling the exam. Each room i has capacity $c(i)$ (number of places of the room). Each day is divided into 4 slots $1, 2, 3, 4$. Each class i has number of students $d(i)$ ($i = 1, \dots, N$). Among N classes, there are K pairs of classes (i, j) in which class i and class j have the same student participating in the exam. It means that these 2 classes cannot be scheduled in the same time-slot. Compute the exam time-table such that the number of days used is minimal.

The data format

- First line: N and M
- Second lines: $d_1, d_2, \dots, d_N$
- Third lines: $c_1, c_2, \dots, c_M$
- Fourth line: K
- Last K lines: i and j (2 classes having a same student registered, these classes cannot be scheduled in the same slot)

Generated Data

The input size is denoted by $(N \times M)$.

- 15 tests:
 (5×2) , (6×2) , (7×3) , (8×3) , (9×4) , (10×3) ,
 (11×4) , (12×4) , (13×4) , (14×4) , (15×4) , (16×4) ,
 (17×4) , (18×5) , (19×5) .
- 10 tests:
 (50×10) , (51×10) , (52×11) , (53×11) , (54×11) ,
 (55×11) , (56×12) , (57×12) , (58×12) , (59×13) .
- 10 tests:
 (100×10) , (200×10) , (300×10) , (400×10) , (500×10) ,
 (600×11) , (700×11) , (800×11) , (900×10) , (1000×10)
- 3 tests mainly for heuristic testing:
 (1000×20) , (2000×20) , (3000×20))

Each test case is randomly generated as follows:

- Each class has a number of students between 20 and 70.
- Let S be the largest number of students in any class. Each room has a capacity between $S/2$ and $3S/2$.
- The number of pairs of classes that share at least one student is at most $N(N + 1)/4$.

2 Modeling the Problem

In the model, let:

- N : the number of classes
- M : the number of rooms
- d_i : the number of students in class i
- c_j : the capacity of room j
- P : the set of conflict pairs (i.e., pairs of classes that share at least one student)

2.1 CP Model

2.1.1 Decision Variables

- $slot_i \in \{1, \dots, N\}$: time-slot assigned to class i
- $room_i \in \{1, \dots, M\}$: room assigned to class i

2.1.2 Constraints

- A class can only be assigned to a room if its number of students does not exceed the room's capacity:

$$d_i > c_j \Rightarrow room_i \neq j$$

- Classes that share at least one student cannot be scheduled in the same time-slot:

$$slot_i \neq slot_j, \forall (i, j) \in P$$

- Only one class per room per time-slot:

$$slot_i = slot_j \Rightarrow room_i \neq room_j$$

2.1.3 Objective Function

Minimize the number of time-slots used:

$$\min \left(\max_{i=1}^N slot_i \right)$$

2.2 MIP Model

2.2.1 Decision Variables

- A class can only be assigned to a room if its number of students does not exceed the room's capacity:

$$x_{i,s,r} = 0 \text{ if } c_j < d_i$$

- Each class is scheduled exactly once:

$$\sum_{s=1}^N \sum_{r=1}^M x_{i,s,r} = 1, \forall i \in \{1, \dots, N\}$$

- Classes that share at least one student cannot be scheduled in the same time-slot:

$$\sum_{\text{valid } r} (x_{i,s,r} + x_{j,s,r}) \leq 1, \forall (i, j) \in P, \forall s \in \{1, \dots, S\}$$

- Only one class per room per time-slot:

$$\sum_{i=1}^N x_{i,s,r} \leq 1, \forall s \in \{1, \dots, S\}, \forall r \in \{1, \dots, M\}$$

- Tracking the latest time-slot used:

$$\sum_{s=1}^N \sum_{r=1}^M (s+1) \cdot x_{i,s,r} \leq D, \forall i \in \{1, \dots, N\}$$

2.2.2 Objective Function

Minimize the number of time-slots used:

$$\min D$$

3 Heuristic Approaches

3.1 Overview

To solve this problem with large size input, we propose these 2 algorithms :

- DSatur algorithm: a graph coloring algorithm based on the concept "degree of saturation"
- Ant Colony algorithm (ACO): an algorithm that simulates the behavior of ant colonies

Custom terms

In this report, the term "cost" refers to the number of slots required to schedule all exams

3.2 DSatur

- **Degree of saturation:**

The degree of saturation of a vertex is the number of different colors assigned to its adjacent vertices.

- **Main Idea:**

DSatur colors the vertices of a graph one after another, adding a previously unused color when needed. After a vertex has been colored, the algorithm determines the next vertex to be colored based on the degree of saturation; the chosen vertex has the highest degree of saturation. If there are multiple vertices with the highest degree of saturation, the vertex with the highest degree is chosen.

- **Apply to ET problem:**

We model the exam scheduling problem as a graph coloring task on a conflict graph $G = (V, E)$, where each vertex represents a class and edges denote conflicts. Using the DSatur algorithm, each class $v \in V$ is assigned the earliest possible time slot t_v not used by its neighbors. A suitable room r_v is then selected; if unavailable, a new slot is chosen for v . This process repeats until all vertices are assigned valid time-slot and room pairs.

- **Process:**

- Initialize all classes as uncolored. For each class, set its saturation degree to 0.
- Select the class with the highest degree (most conflicts) to color first and assign it slot 1.
- Mark this slot as used by all of its neighbors. Increase their saturation degrees.
- For the remaining uncolored classes:
 - * Select the class with the highest saturation degree. Break ties by picking the one with the highest degree.
 - * Assign the smallest slot not used by its neighbors.
 - * Update the neighbor colors and saturation degrees accordingly.
- After all slots are assigned, try to assign a suitable room (with sufficient capacity and not yet used in that slot).
- If no room is available in the assigned slot, iterate over other slots (starting from 0), check conflict-free condition, and try to assign a new slot and room.

- **Pseudo code:**

Algorithm 1 DSATUR Algorithm

```
1: procedure DSATUR( $G = (V, E)$ )
2:   Input: Graph  $G = (V, E)$ 
3:   Output: Vertex colors  $c(v)$  for each  $v \in V$ 
4:    $C \leftarrow \emptyset$ ;  $U \leftarrow V$ ; compute  $deg_G(U)$ 
5:   Select one vertex  $v$  randomly with  $\max_{v \in U} \{deg_G(U)(v)\}$ 
6:    $c(v) \leftarrow 1$ ;  $C \leftarrow C \cup \{v\}$ ;  $U \leftarrow U \setminus \{v\}$ 
7:   Update  $dsatur_G(C)$  and  $deg_G(U)$ 
8:   repeat
9:     Find a vertex  $v$  with  $\max_{v \in U} \{dsatur_G(C)(v)\}$ 
10:    if there is a subset  $U' \subseteq U$  of vertices with the same max saturation degree then
11:      Select one vertex  $v$  randomly with  $\max_{v \in U'} \{deg_G(U)(v)\}$ 
12:    end if
13:    Find the least possible color  $k$  that can color the selected vertex  $v$ 
14:     $c(v) \leftarrow k$ ;  $C \leftarrow C \cup \{v\}$ ;  $U \leftarrow U \setminus \{v\}$ 
15:    Update  $dsatur_G(C)$  and  $deg_G(U)$ 
16:  until  $U = \emptyset$ 
17: end procedure
```

3.3 ACO

- **Main Idea:**

The ACO algorithm simulates the path-finding behavior of real ant colonies. Artificial ants explore the solution space and deposit pheromones along their paths. Paths with higher pheromone levels become more attractive to other ants, gradually guiding the colony toward optimal solutions through a process of pheromone reinforcement and evaporation.

- **Apply to ET problem:**

We start by initializing pheromone levels for class-timeslot assignments. Each "ant" builds a schedule by assigning slots and rooms based on pheromones and conflict heuristics. The schedule's cost is evaluated. Pheromones are updated using the best solution. This process repeats for several iterations to optimize.

- **Process :**

1. **Initialization:**

- Initialize pheromone trails $\tau_{i,s}$ for all class-slot pairs (i, s) with a default value (e.g., 1.0).
- Set the global best cost to a large value.

2. **Solution Construction (for each ant):**

- For each class i , evaluate all available time slots s :
 - * Check whether a room is available at slot s that can accommodate class i .
 - * Compute the number of conflicts if class i is placed at slot s .
 - * Compute the desirability value as:

$$\text{value}_{i,s} = (\tau_{i,s})^\alpha \cdot \left(\frac{1}{1 + \text{conflicts}} \right)^\beta$$

- Select the slot with the highest desirability and assign the earliest available valid room.
- Repeat until all classes are scheduled.

3. **Pheromone Update:**

- Evaporate all pheromone trails:

$$\tau_{i,s} \leftarrow (1 - \rho) \cdot \tau_{i,s}$$

- Identify the best ant k^* in the current iteration (having the lowest number of slot conflicts).
- Reinforce the pheromone on its used class-slot pairs:

$$\tau_{i,s} \leftarrow \tau_{i,s} + \frac{Q}{1 + \text{cost}_{k^*}}$$

- If this ant improves the global best solution, update the best slot-room assignment.

4. **Repeat:** Repeat the construction and update steps for a fixed number of iterations or until convergence.

- **Parameter Settings:**

The performance of the Ant Colony Optimization (ACO) algorithm is governed by several hyperparameters that influence the balance between exploration and exploitation, as well as the convergence speed of the algorithm. The key parameters used in our implementation are:

- α : Importance of pheromone trails.
- β : Importance of conflict avoidance.
- ρ : Pheromone evaporation rate.
- Q : Pheromone deposit amount.
- **Max_ants**: Number of ants per iteration.
- **Max_loops**: Number of total iterations.

Different parameter settings may lead to results of varying quality. We tested several configurations and identified the optimal values, as shown below :

Table 1: Experimental Settings and Optimal Values

Parameter	Test Values	Optimal Value
α	0.5, 1.2, 2.0	1.0
β	1.0, 2.2, 2.5, 3.0	2.0
ρ	0.05, 0.08, 0.15, 0.2, 0.3	0.1
Q	300, 400, 600, 800	500
Max_ants	6, 7, 10, 12, 15, 100	9
Max_loops	80, 100, 120, 150, 200	100

- **Pseudo code:**

Algorithm 2 ACO Algorithm

```
1: Initialize the pheromone trails and parameters;  
2: while not termination do  
3:   for each ant completes a tour do  
4:     LocalPheromoneUpdate();  
5:   end for  
6:   UpdateBest();  
7:   GlobalPheromoneUpdate();  
8: end while
```

4 Result and Analysis

4.1 Result

4.1.1 CP model and MIP model solver

Exact Solution

Input sizes		CP		MIP	
N	M	f	t (ms)	f	t (ms)
5	2	5	7	5	35
6	2	3	12	3	85
7	3	3	18	3	162
8	3	3	17	3	278
9	4	4	27	4	123
10	3	4	26	4	164
11	4	6	31	6	423
12	4	4	43	4	577
13	4	5	142	5	902
14	4	8	93	8	1334
15	4	9	72	9	28636
16	4	8	173	8	21724
17	4	10	188	F	F
18	5	7	189	F	F
19	5	8	13841	F	F
50	10	5	7345	F	F
51	10	F	F	F	F
52	11	F	F	F	F
53	11	F	F	F	F

Table 2: Results of exact solutions for CP and MIP solvers

All Solution

As a result of the large size and intricate nature of the optimization problems under consideration, we were only able to obtain feasible solutions for test instances with input size smaller than or equal to (53×10) for the CP solver, and (16×4) for the MIP solver, within a time limit of 300 seconds. This indicates that the optimization problems are highly complex and

computationally demanding, requiring significant computational resources to solve. To ensure the reliability of our findings, we conducted ten runs of each solver and computed the average results, thereby mitigating the potential impact of random variations and ensuring the robustness of our numerical experiments.

Input sizes		CP				MIP			
N	M	f_{min}	f_{max}	f_{avg}	$t(ms)_{avg}$	f_{min}	f_{max}	f_{avg}	$t(ms)_{avg}$
5	2	5	5	5	7	5	5	5	35
6	2	3	3	3	12	3	3	3	85
7	3	3	3	3	18	3	3	3	162
8	3	3	3	3	17	3	3	3	278
9	4	4	4	4	27	4	4	4	123
10	3	4	4	4	26	4	4	4	164
11	4	6	6	6	31	6	6	6	423
12	4	4	4	4	43	4	4	4	577
13	4	5	5	5	142	5	5	5	902
14	4	8	8	8	93	8	8	8	1334
15	4	9	9	9	72	9	9	9	28636
16	4	8	8	8	173	8	8	8	21724
17	4	10	10	10	188	10	10	10	301973
18	5	7	7	7	189	7	7	7	303064
19	5	8	8	8	13841	8	8	8	302770
50	10	5	5	5	7345	5	5	5	321161
51	10	7	8	7.33	300154	9	9	9	325693
52	11	8	9	8.33	300164	15	15	15	325189
53	11	14	14	14	300537	15	15	15	315767
54	11	9	11	10	300218	9	11	10.33	316766
55	11	14	14	14	300202	15	15	15	314033
56	12	9	10	9.67	300191	13	15	14	326837
57	12	9	12	11	300184	18	18	18	337534
58	12	9	11	10.33	300222	23	27	25.33	339576
59	13	9	9	9	300262	17	17	17	342626
100	10	18	18	18	300696	F	F	F	F
200	10	183	210	195.33	302356	F	F	F	F
300	10	F	F	F	F	F	F	F	F
400	10	F	F	F	F	F	F	F	F
500	10	F	F	F	F	F	F	F	F
600	11	F	F	F	F	F	F	F	F
700	11	F	F	F	F	F	F	F	F
800	11	F	F	F	F	F	F	F	F
900	10	F	F	F	F	F	F	F	F
1000	10	F	F	F	F	F	F	F	F
1000	20	F	F	F	F	F	F	F	F
2000	20	F	F	F	F	F	F	F	F
3000	20	F	F	F	F	F	F	F	F

Table 3: Average results of CP solver and MIP solver

Heuristic method

Input sizes		DSatur				ACO			
N	M	f_{min}	f_{max}	f_{avg}	$t(ms)_{avg}$	f_{min}	f_{max}	f_{avg}	$t(ms)_{avg}$
5	2	5	5	5	2	5	5	5	1
6	2	4	4	4	2	4	4	4	1
7	3	4	4	4	3	4	4	4	1
8	3	3	3	3	3	4	4	4	1
9	4	6	6	6	4	5	5	5	1
10	3	4	4	4	3	4	4	4	1
11	4	8	8	8	4	7	7	7	2
12	4	5	5	5	4	6	6	6	2
13	4	8	8	8	4	8	8	8	2
14	4	9	9	9	5	9	9	9	3
15	4	11	11	11	5	11	11	11	3
16	4	9	9	9	5	9	9	9	4
17	4	10	10	10	6	11	11	11	5
18	5	7	7	7	6	9	9	9	6
19	5	9	9	9	6	10	10	10	6
50	10	7	7	7	16	11	11	11	68
51	10	9	9	9	16	11	11	11	68
52	11	9	9	9	17	11	11	11	73
53	11	15	15	15	18	15	15	15	76
54	11	13	13	13	18	13	13	13	73
55	11	18	18	18	17	17	17	17	72
56	12	9	9	9	17	12	12	12	87
57	12	10	10	10	18	12	12	12	90
58	12	13	13	13	18	13	13	13	92
59	13	10	10	10	18	12	12	12	108
100	10	22	22	22	29	23	23	23	434
200	10	34	34	34	59	35	35	35	2837
300	10	52	52	52	91	53	53	53	9329
400	10	68	68	68	123	68	68	68	34071
500	10	69	69	69	159	74	74	74	43301
600	11	100	100	100	210	95	95	95	56639
700	11	87	87	87	228	93	93	93	87889
800	11	167	167	167	416	163	163	163	126817
900	10	134	134	134	366	131	131	131	190073
1000	10	207	207	207	589	187	187	187	252867
1000	20	116	116	116	355	127	127	127	277634
2000	20	169	169	169	898	F	F	F	F
3000	20	201	201	201	2152	F	F	F	F

Table 4: Average results of DSatur algorithm and ACO algorithm

4.2 Analysis

4.2.1 Exact solution

Our evaluation of the accuracy of each algorithm revealed that CPModel and MIP were capable of generating exact solutions for a subset of test cases :

- CP was able to produce exact solutions for tests with sizes :
(5×2), (6×2), (7×3), (8×3), (9×4),
(10×3), (11×4), (12×4), (13×4), (14×4),
(15×4), (16×4), (17×4), (18×5), (19×5),
(50×10), (51×10), (52×11), (53×11)
- MIP was only able to produce exact solutions for test cases with sizes :
(5×2), (6×2), (7×3), (8×3), (9×4),
(10×3), (11×4), (12×4), (13×4), (14×4),
(15×4), (16×4)

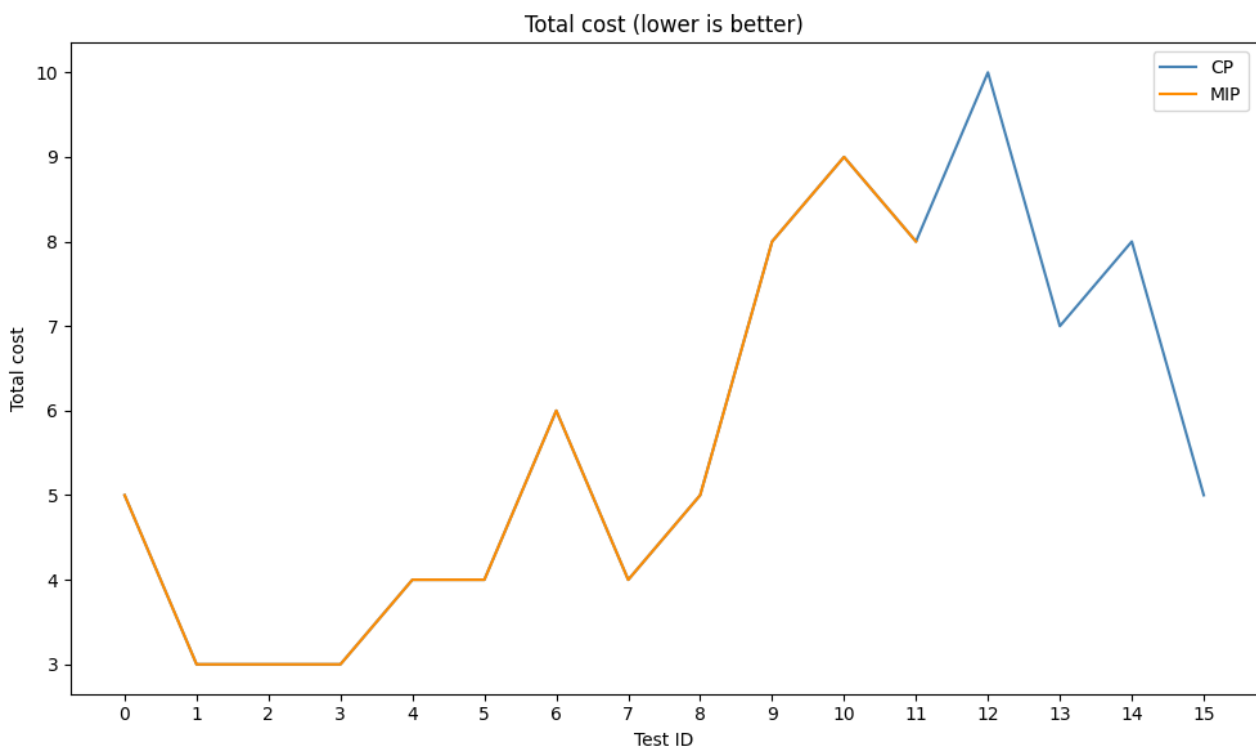


Figure 1: Total Cost Comparison: CP vs MIP

4.2.2 All Solutions

After conducting experiments on the described four algorithms, we come up with some conclusions:

- CP is unable to handle datasets larger than (200×10) .
- MIP is unable to handle datasets larger than (59×13) .
- DSatur is capable of handling all test cases.
- ACO fails to solve two test cases: (2000×20) and (3000×20) .

About total costs, from the experiments, we conclude that :

- MIP performs the worst.
- CPModel and MIP generate nearly equivalent results on small test cases.
- DSatur and ACO exhibit very similar performance for all test cases with sizes smaller than (1000×20) . These two algorithms both outperform CPModel and MIP for test cases with sizes larger than (200×10) . However, for test IDs > 36 , DSatur maintains its efficiency, while ACO fails to solve these test cases.

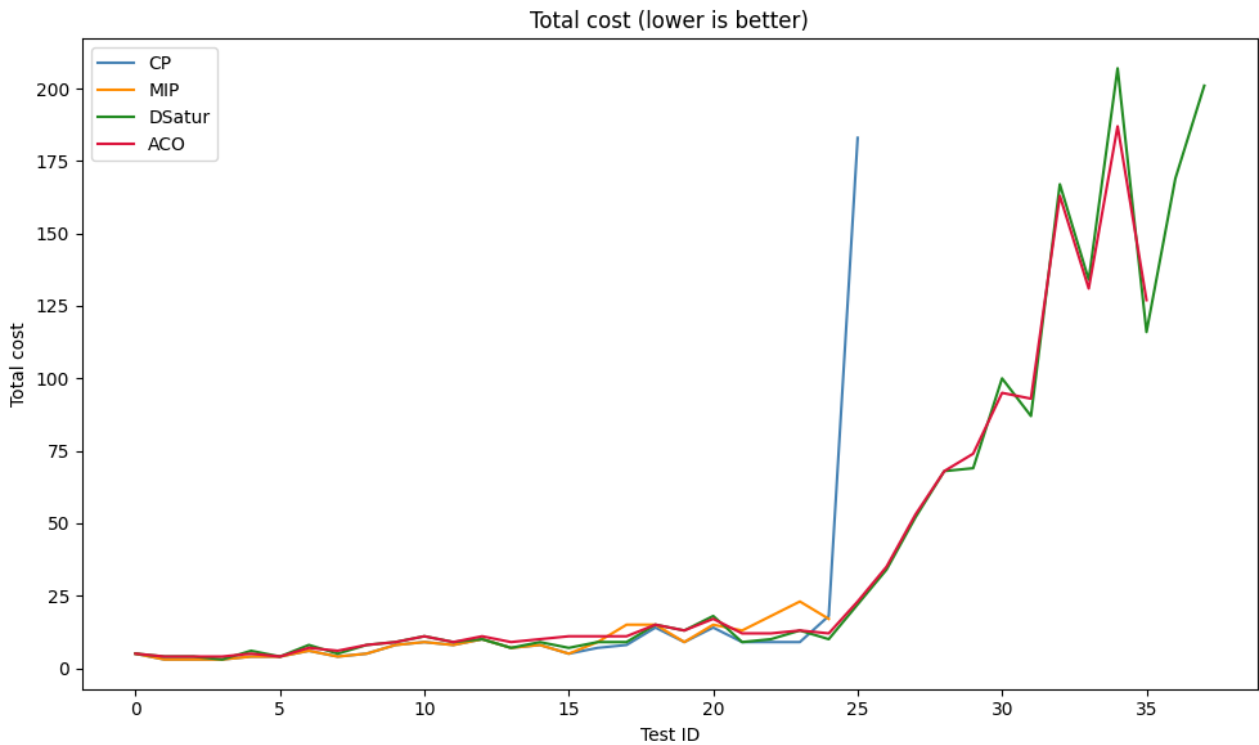


Figure 2: Total Cost Comparison of all Solutions

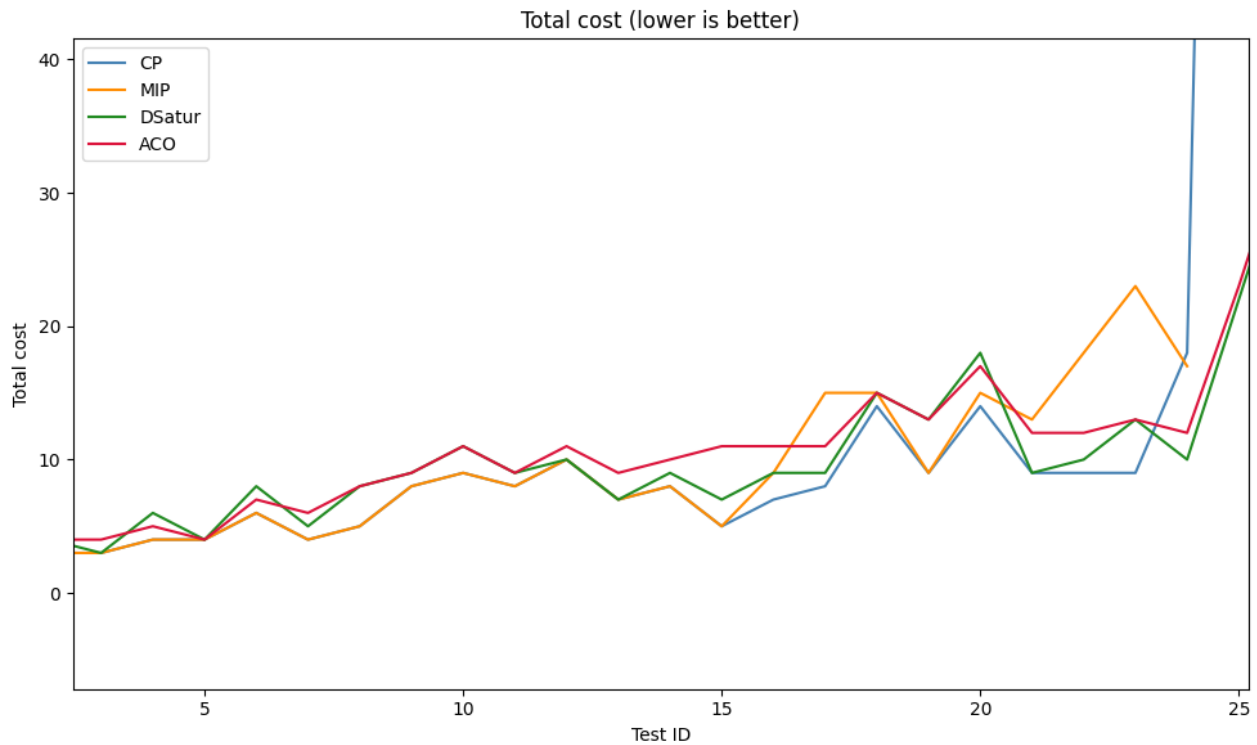


Figure 3: Zoomed Total Cost Comparision of all Solutions for the first 25 test case

4.2.3 Runtime

The running time of each algorithm was also evaluated , and our experiments indicate that :

- MIP reaches time limit of 300 seconds for all tests with size greater than or equal ($17 \times 4 \times 113$).
- CPModel reaches time limit of 300 seconds for allmost testcase with size greater than or equal ($300 \times 10 \times 22415$).
- ACO reaches time limit of 300 seconds for all tests with size greater than or equal ($2000 \times 20 \times 823453$).
- DSatur demonstrates the capability to solve all test cases regardless of their size, achieving remarkable efficiency, even for the largest test case of ($3000 \times 20 \times 1010240$).

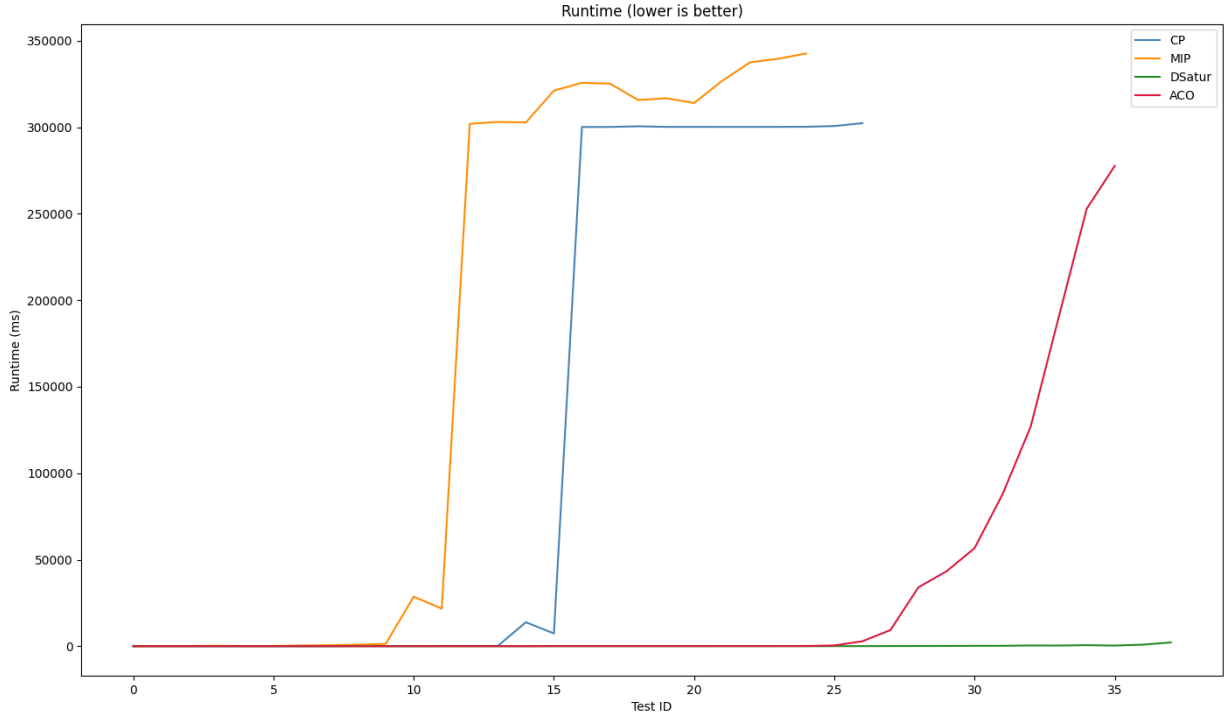


Figure 4: Runtime of all Solutions

5 Conclusion

In conclusion, based on our evaluation, the heuristic methods demonstrate both high accuracy and efficiency, making them a valuable tool for practical applications that require timely and reliable results. Meanwhile, the CP-SAT solver may be suitable for small datasets due to its reasonable performance on limited scales. In contrast, the MIP solver is not recommended, as it exhibits significantly poorer performance in both accuracy and computational efficiency.

6 Limitations and Future research

Throughout the experimental process, we also identified some limitations and opportunities for future research :

At first, our study only considered a specific set of test cases, with insufficient exploration of the effects of varying problem sizes . Therefore, future research could focus on evaluating the performance of the algorithms on a wider range of test cases.

Additionally, Future research could explore the impact of parameter settings on heuristic algorithms, such as Ant Colony Optimization (ACO). During our experimental process, we observed that certain parameter values led to bad outcomes. For instance, we initially hypothesized that increasing the number of ants would improve solution quality. However, this

resulted in poorer performance due to randomness and a phenomenon known as "pheromone overload." This effect caused excessive pheromone accumulation on bad solutions, disrupting the search process and reducing the algorithm's focus on optimal solutions. We believe that with carefully selected parameter values, more effective solutions can be achieved.

Furthermore, future research could focus on enhancing the performance of the Heuristic algorithm while maintaining its efficiency, or explore the potential for hybrid algorithms that combine elements of the CP, MIP, and Heuristic methods to achieve even better outcomes. For example, DSatur algorithm demonstrates the superpriority in accuracy and time efficiency. It can produce a high-quality solution with remarkably shorter runtimes compare to most other algorithms . We believe that, with further improvements , DSatur will be a promising candidate for solving ET-related optimization problems.

7 Visualization

In terms of visualization, we generated the timeslot-room tables corresponding to the results produced by the CP solver, MIP solver and the Heuristic solver with a testcase. These tables illustrate the differences in scheduling performance among 4 methods.

Testcase : (10 × 3)

```
10 3
72 77 71 71 53 45 53 53 66 70
79 53 70
16
1 2
1 3
1 8
1 10
2 5
2 9
3 6
3 9
4 10
5 8
5 10
7 8
7 9
7 10
8 9
9 10
```

Class	Room 1	Room 2	Room 3
1	Slot 1		
2	Slot 4		
3	Slot 3		
4	Slot 2		
5		Slot 1	
6		Slot 2	
7		Slot 4	
8			Slot 2
9			Slot 1
10			Slot 3

(a) CP-Sat

Class	Room 1	Room 2	Room 3
1	Slot 2		
2	Slot 4		
3	Slot 1		
4	Slot 3		
5		Slot 3	
6		Slot 2	
7		Slot 1	
8			Slot 2
9			Slot 3
10			Slot 4

(b) MIP

Class	Room 1	Room 2	Room 3
1	Slot 1		
2	Slot 2		
3	Slot 3		
4	Slot 4		
5		Slot 1	
6			Slot 1
7		Slot 3	
8		Slot 2	
9			Slot 4
10			Slot 2

(c) DSatur

Class	Room 1	Room 2	Room 3
1	Slot 1		
2	Slot 2		
3	Slot 3		
4	Slot 4		
5		Slot 1	
6			Slot 1
7		Slot 2	
8		Slot 3	
9			Slot 4
10			Slot 3

(d) ACO

Table 5: Exam Schedules Generated by Different Algorithms (Max 4 Slots)

Slot 1
 Slot 2
 Slot 3
 Slot 4

References

- [1] GeeksforGeeks.
DSatur Algorithm for Graph Coloring.
Available at: <https://www.geeksforgeeks.org/dsatur-algorithm-for-graph-coloring/>.
- [2] Ahmad Firdaus Khair, Mokhairi Makhtar, Munirah Mazlan, Mohamad Afendee Mohamed, and Mohd Nordin Abdul Rahman.
An Ant Colony Algorithm for Universiti Sultan Zainal Abidin Examination Timetabling Problem.
Indonesian Journal of Electrical Engineering and Computer Science, Vol. 13, No. 2, pp. 769–775, 2019.
- [3] Masri Ayob, Ariff Md. Ab. Malik, Salwani Abdullah, and Abdul Razak Hamdan.
Solving a Practical Examination Timetabling Problem: A Case Study.
In: Orgun M.A., Thornton J. (eds) *Advances in Artificial Intelligence. AI 2007. Lecture Notes in Computer Science*, vol 4830. Springer, Berlin, Heidelberg.